

## **Lesson 5: ML concepts for multiomics**

# Second (ML) part of multiomics course

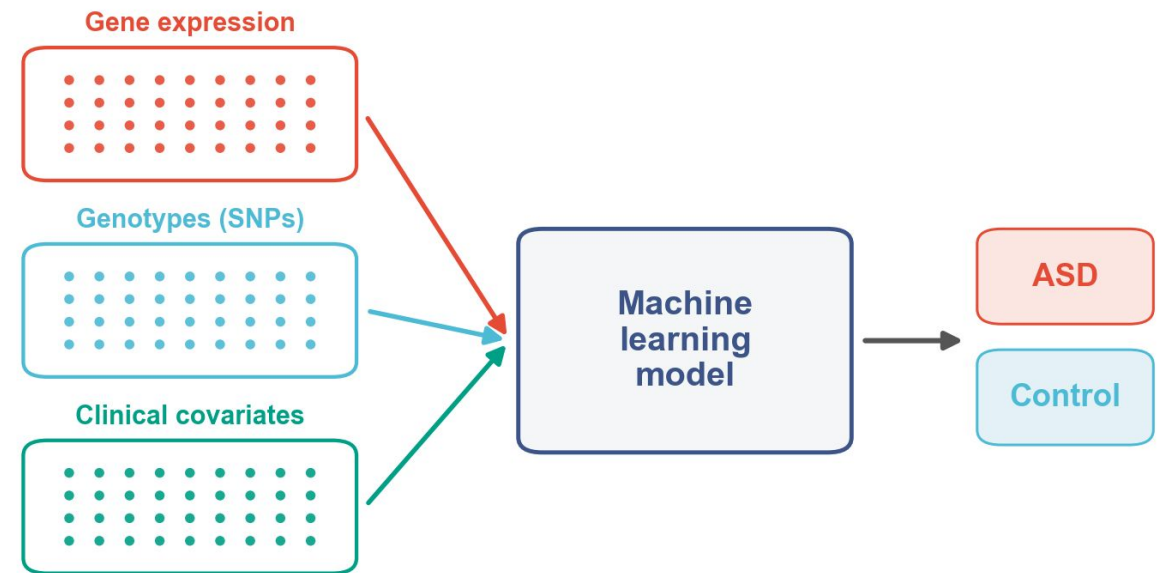
---

5. Basic ML concepts in multiomics
6. Feature selection and biomarker discovery
7. Feature importance and measurement of influence
8. Clustering and subgroup discovery

# Lesson 5 Roadmap

---

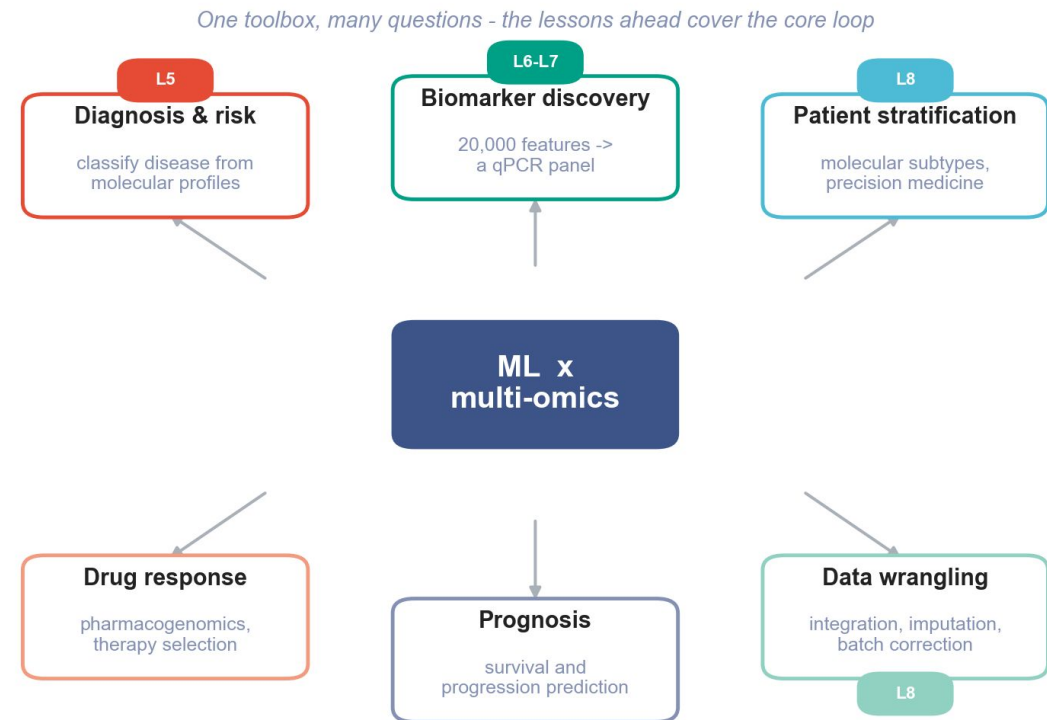
- Core concepts: learning paradigms, data splitting, validation
- Model performance metrics for biology
- LazyPredict: automated model screening
- Google's TabFM - foundation model for tabular data
- Hands-on: ASD multi-omics dataset



*Multi-omics layers → one predictive model → diagnosis*

# Why ML? The Multi-Omics Application Map

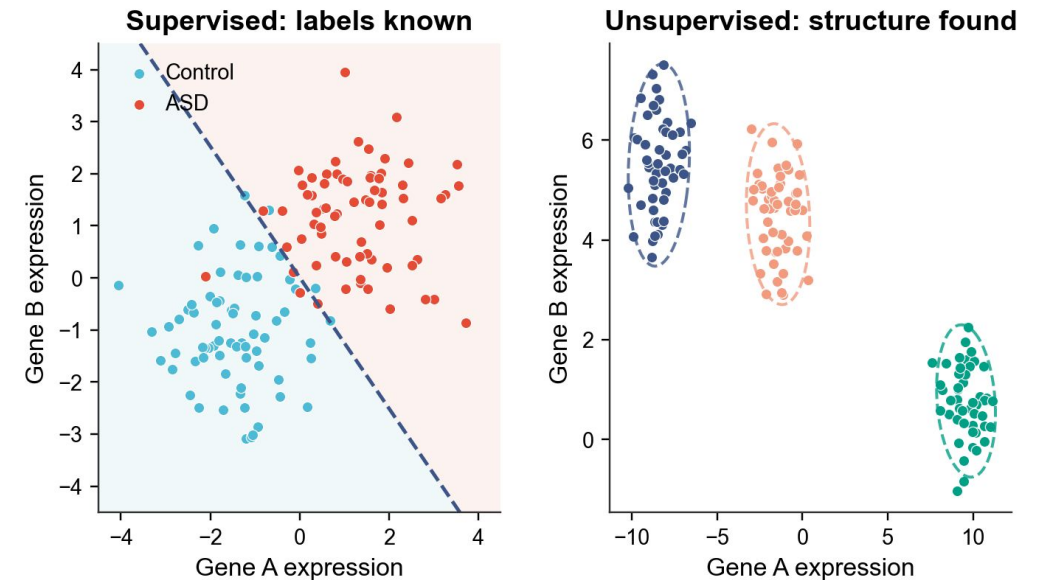
- Diagnosis & risk: classify disease from molecular profiles
- Biomarker discovery: from 30,000 features to a qPCR panel
- Patient stratification: molecular subtypes → precision medicine
- Drug response: pharmacogenomics, therapy selection
- Data wrangling: integration, imputation, batch correction



# 1.1 Supervised vs Unsupervised Learning

---

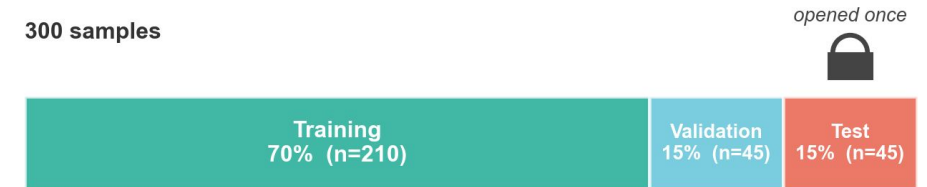
- Supervised: learn from labeled examples ( $X \rightarrow y$ )
  - Classification: ASD vs control
  - Regression: predict IQ
- Unsupervised: find structure without labels
  - Clustering: patient subgroups
  - PCA/UMAP: visualization
- Multi-omics workflows typically combine both
- MOFA (Lesson 4) was unsupervised; today we go supervised



## 1.2 Training, Validation and Test Sets

---

- Training set (~70-80%): model learns its parameters
- Validation set (~10-20%): choose hyperparameters
- Test set (~10-20%): touched ONCE, at the very end
- Stratified splitting preserves case/control proportions
- Golden rule: never let test data influence any decision
- Small cohorts: cross-validation replaces the validation set



Stratified split - class ratio preserved (2:1 ASD : Control):



# 1.3 Cross-Validation

---

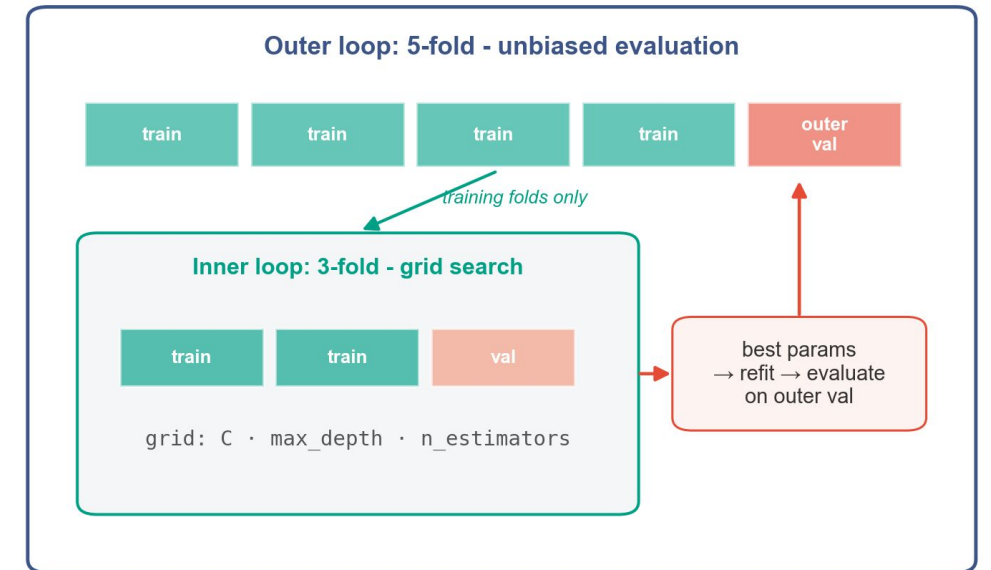
- K-fold CV: split training data into K parts (folds)
- Train on K-1 folds, validate on the remaining fold; repeat K times
- Report mean  $\pm$  SD  $\rightarrow$  performance AND its uncertainty
- Stratified K-fold: every fold keeps class proportions
- Our notebook (152 raw features): CV AUC  $0.87 \pm 0.06$
- Small biology cohorts: CV squeezes maximum use out of every sample



# 1.4 Nested Cross-Validation

---

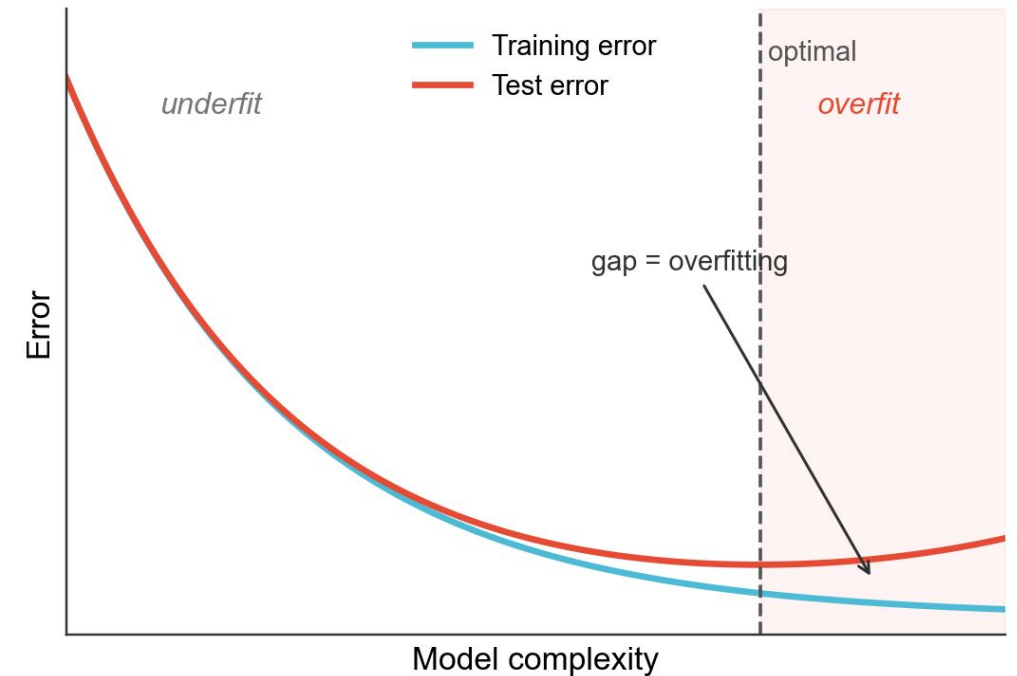
- Problem: tuning hyperparameters on CV folds biases the estimate
- Solution: two loops
  - Inner loop: grid-search hyperparameters (3-fold)
  - Outer loop: unbiased performance estimate (5-fold)
- Used in our ASD notebook to compare LogReg / RF / XGBoost
- Costs more compute - but the estimate you report is honest



# 1.5 Overfitting: The Enemy

---

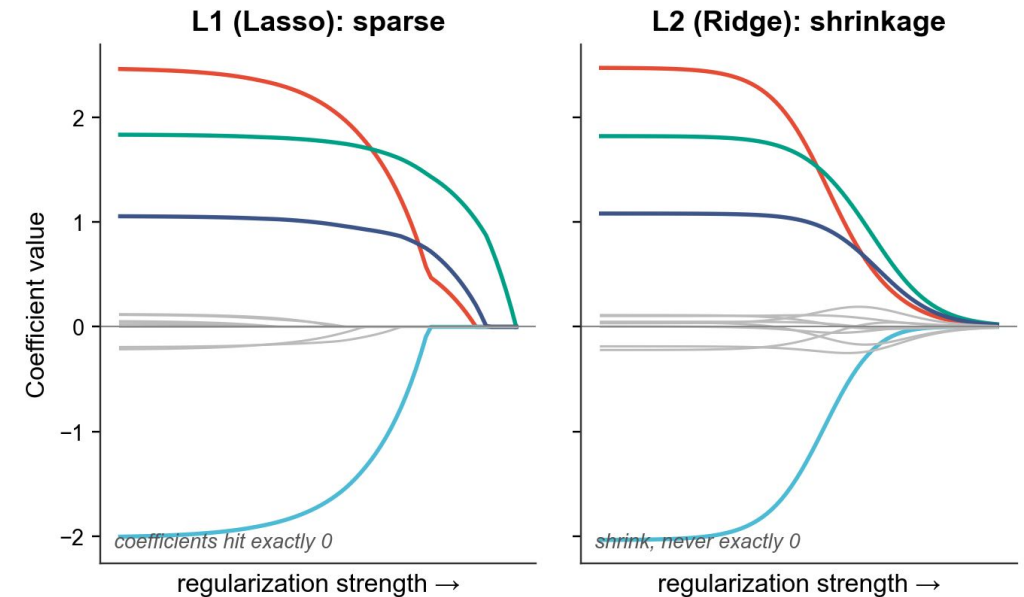
- Model memorizes training data instead of learning biology
- Symptom: perfect training score, poor test score
- Omics is overfitting paradise:  $p \gg n$  (20,000 genes, 300 samples)
- With enough genes, some separate classes purely by chance
- Detection: monitor the train-test performance gap
- Overfitting = memorizing the textbook, then failing the exam



# 1.6 Regularization: The Cure

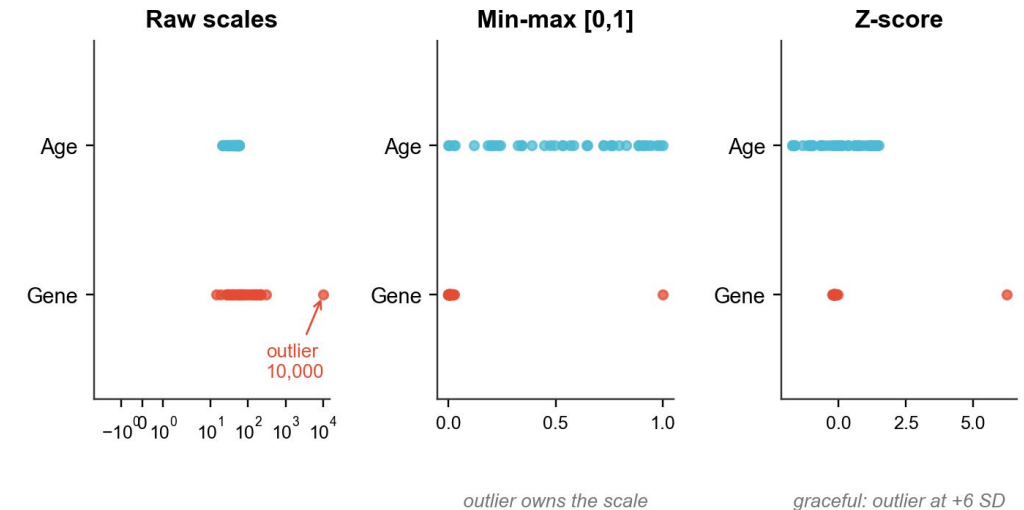
---

- Add a penalty for model complexity to the loss function
- L1 (Lasso): drives coefficients to exactly zero → feature selection
- L2 (Ridge): shrinks all coefficients smoothly → stability
- ElasticNet: L1 + L2 - good default for correlated genes
- Trees: limit `max_depth`, `min_samples_leaf`, `n_estimators`
- Regularization strength = hyperparameter → tune it with CV



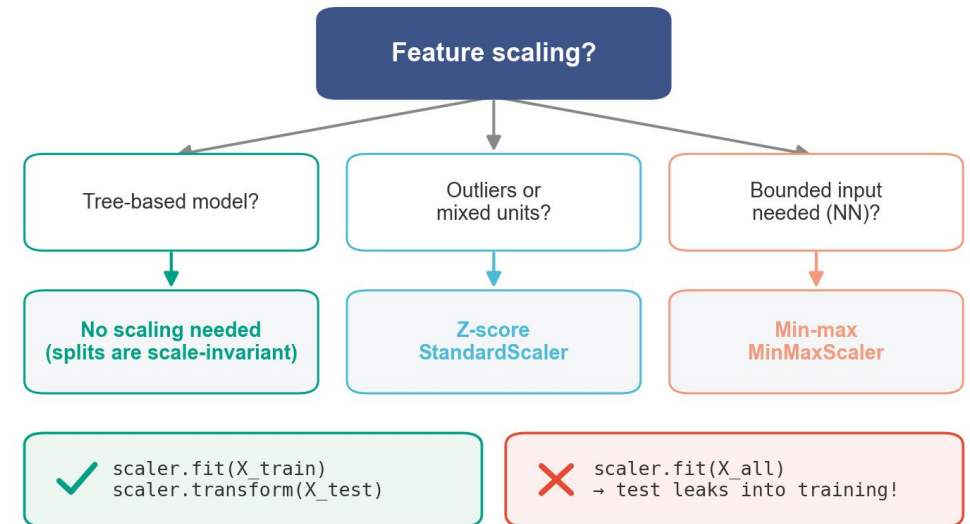
# 1.7 Standardization vs Normalization

- Standardization (z-score):  $x' = (x - \text{mean}) / \text{SD} \rightarrow \text{mean } 0, \text{SD } 1$
- Normalization (min-max):  $x' = (x - \text{min}) / (\text{max} - \text{min}) \rightarrow \text{range } [0, 1]$
- Example: Age 20-60 yrs, gene TPM 0-10,000 - raw scales dominate
- Outliers crush min-max: one extreme sample squeezes all others
- Z-score keeps outlier information; min-max gives bounded inputs
- Comparing TPM to age in years = comparing apples to centimeters



# 1.8 Scaling: Which One, When - and How

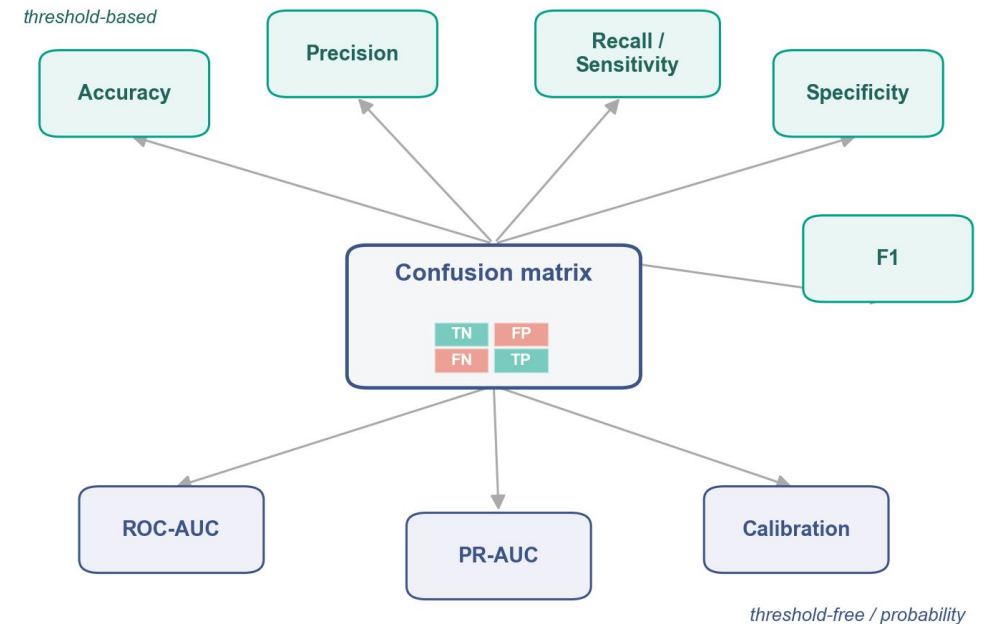
- Use z-score when: outliers present (omics: always), mixed units,
  - distance/penalty-based models (SVM, KNN, Lasso, LogReg)
- Use min-max when: bounded input needed (neural nets, images)
- Trees don't care - splits are scale-invariant
- CRITICAL: fit scaler on training data only, then transform test
- Fitting on all data = leakage → optimistic results



## 2. Model Performance Metrics

---

- One number is never enough - every metric hides something
- Foundation: the confusion matrix (TP, TN, FP, FN)
- Derived: accuracy, precision, recall, specificity, F1
- Threshold-free: ROC-AUC, precision-recall AUC
- Honesty check: calibration
- Choice of metric = clinical question in disguise



## 2.1 The Confusion Matrix

---

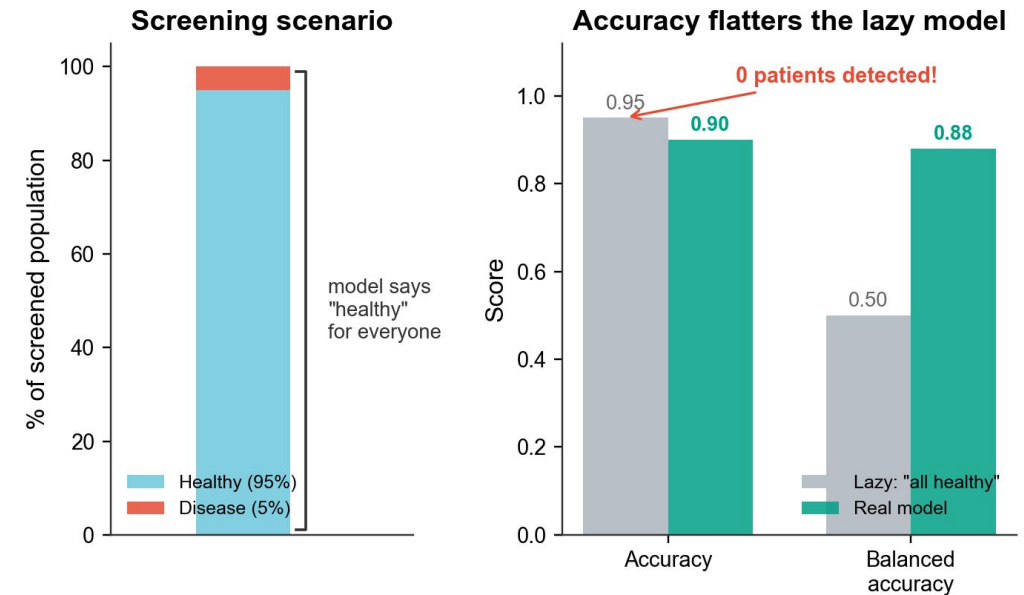
- Rows = truth, columns = prediction (sklearn convention)
- TP: sick called sick    TN: healthy called healthy
- FP: healthy called sick (false alarm)
- FN: sick called healthy (missed case!)
- ASD notebook test set (XGBoost): TN=22, FP=8, FN=3, TP=57
- Every downstream metric is arithmetic on these four cells

**ASD test set (n = 90)**

|        |         |                            |                            |
|--------|---------|----------------------------|----------------------------|
| Actual | Control | <b>22</b><br>True Negative | <b>8</b><br>False Positive |
|        | ASD     | <b>3</b><br>False Negative | <b>57</b><br>True Positive |
|        |         | Control                    | ASD                        |
|        |         | missed cases               | Predicted                  |

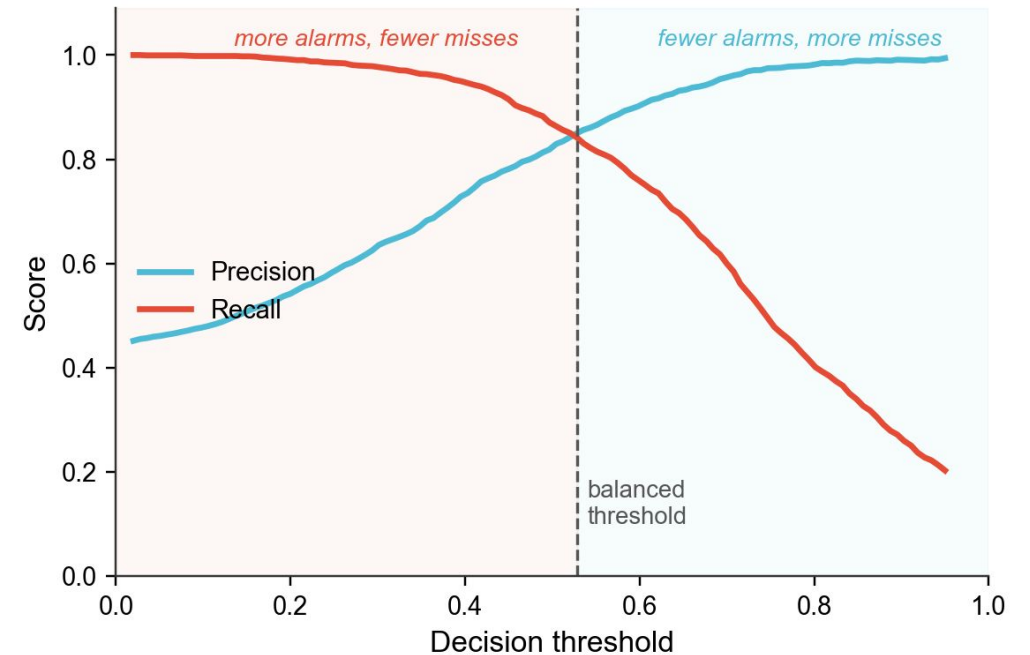
## 2.2 Accuracy - and Why It Fails

- Accuracy =  $(TP + TN) / \text{all predictions}$
- Screening scenario: 5% disease prevalence
- Lazy model 'everyone is healthy' → 95% accuracy, 0 patients found
- Imbalance is the norm in biology, not the exception
- Better: balanced accuracy =  $(\text{sensitivity} + \text{specificity}) / 2$
- Never report accuracy alone for clinical models



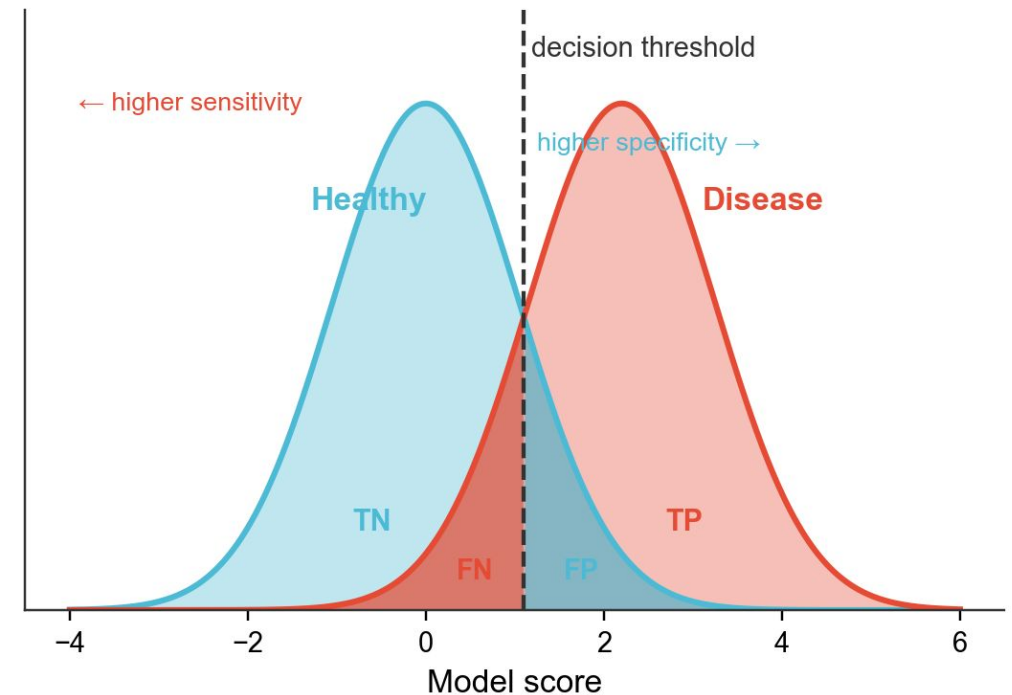
## 2.3 Precision and Recall

- Precision =  $TP / (TP + FP)$ : of my positive calls, how many are right?
- Recall =  $TP / (TP + FN)$ : of the true patients, how many did I find?
- High precision → few false alarms (costly follow-up tests)
- High recall → few missed cases (dangerous diseases)
- They trade off: move the threshold, one rises, the other falls
- ASD notebook: precision 0.88, recall 0.95 on test set



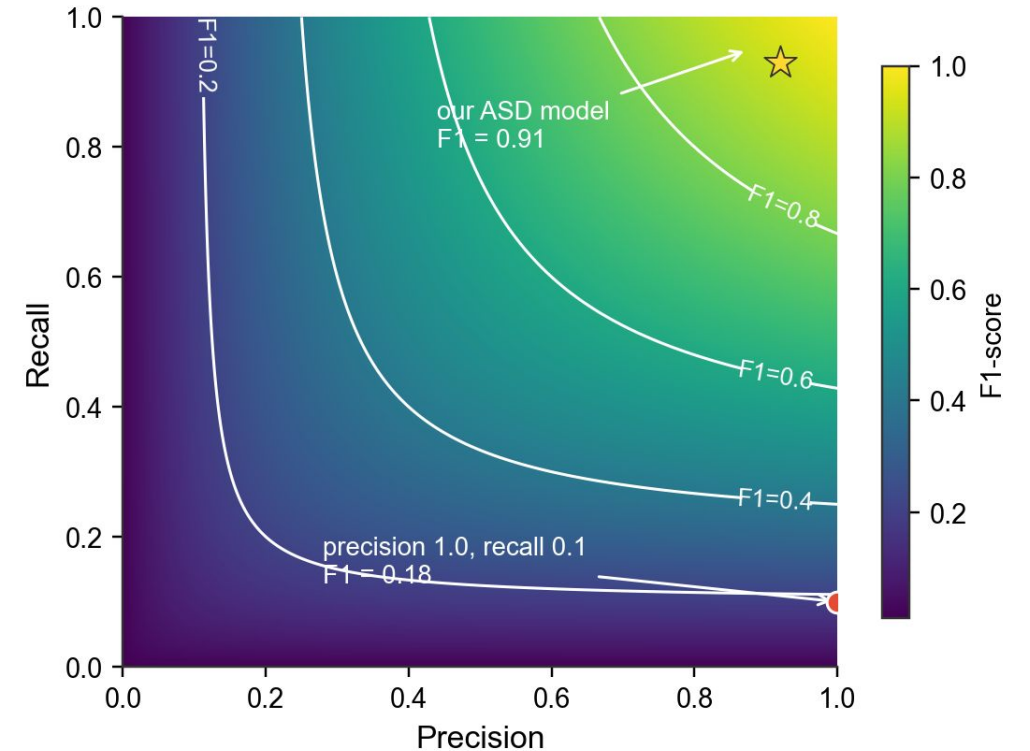
## 2.4 Sensitivity and Specificity

- Sensitivity (= recall):  $TP / (TP + FN)$  - ability to detect disease
- Specificity:  $TN / (TN + FP)$  - ability to clear the healthy
- Screening test: maximize sensitivity (never miss a case)
- Confirmatory test: maximize specificity (never mislabel healthy)
- The threshold slides along two overlapping distributions
- Same math as diagnostics you already know (PCR, ELISA cut-offs)



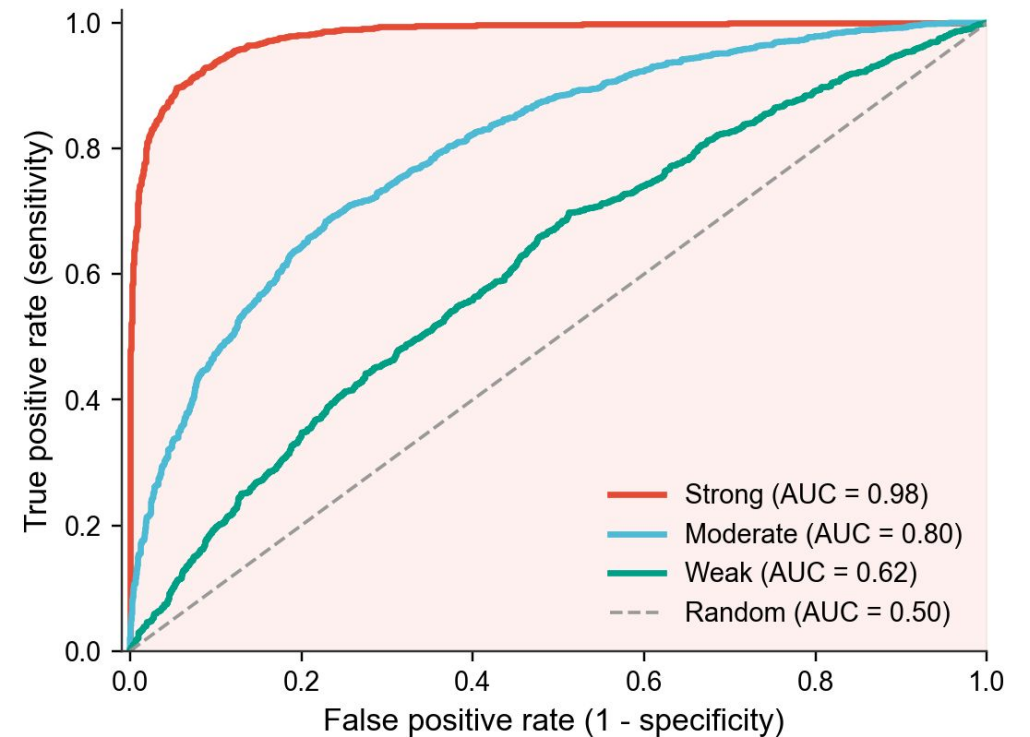
## 2.5 F1-Score: One Number for the Trade-off

- $F1 = 2 \cdot (\text{precision} \cdot \text{recall}) / (\text{precision} + \text{recall})$
- Harmonic mean: brutally punishes imbalance between the two
- Precision 1.0 with recall 0.1  $\rightarrow F1 = 0.18$  (not 0.55!)
- Useful for imbalanced data and model comparison
- Blind spot: ignores true negatives entirely
- ASD notebook:  $F1 = 0.91$  for the winning model



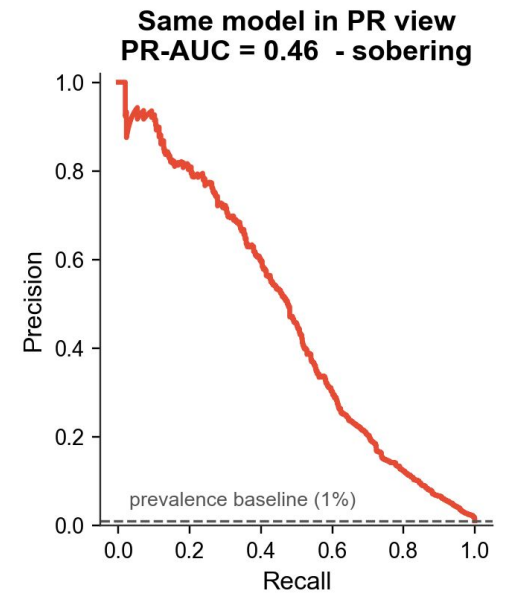
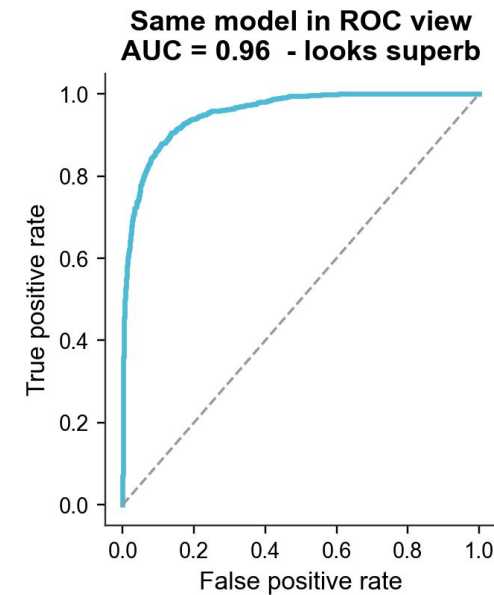
## 2.6 ROC Curve and AUC

- ROC: sensitivity vs (1 - specificity) across ALL thresholds
- AUC = area under the curve - threshold-free summary
- AUC 1.0 perfect • 0.5 coin flip •  $<0.5$  flip your labels
- Interpretation:  $P(\text{random case scores above random control})$
- Robust to class imbalance in ranking terms - but see next slide
- ASD notebook, 152 raw features: XGBoost 0.92, LogReg 0.83



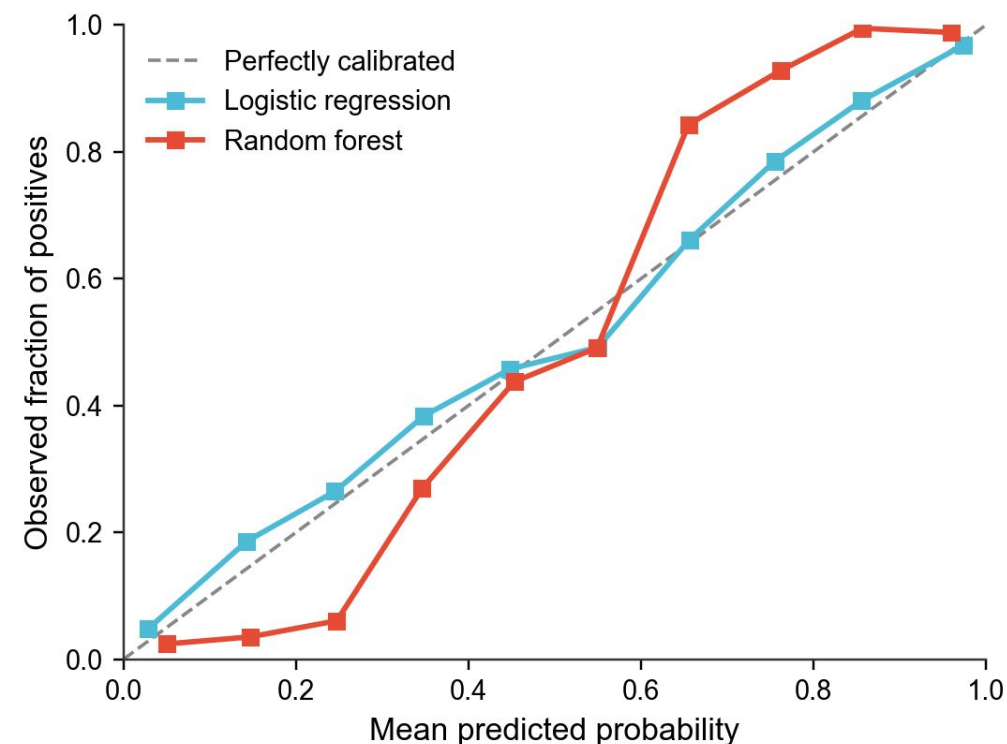
## 2.7 Precision-Recall Curve: Rare-Disease Reality

- PR curve: precision vs recall across thresholds
- Baseline = prevalence (not 0.5!) - brutal at 1% disease
- ROC can look great while precision is terrible
- 10,000 people, 1% sick, sens 90% / spec 90% → precision only 8%
- Rare positives (rare disease, rare cell type): report PR-AUC
- Average precision (AP) = area under the PR curve



## 2.8 Calibration: Can You Trust the Probabilities?

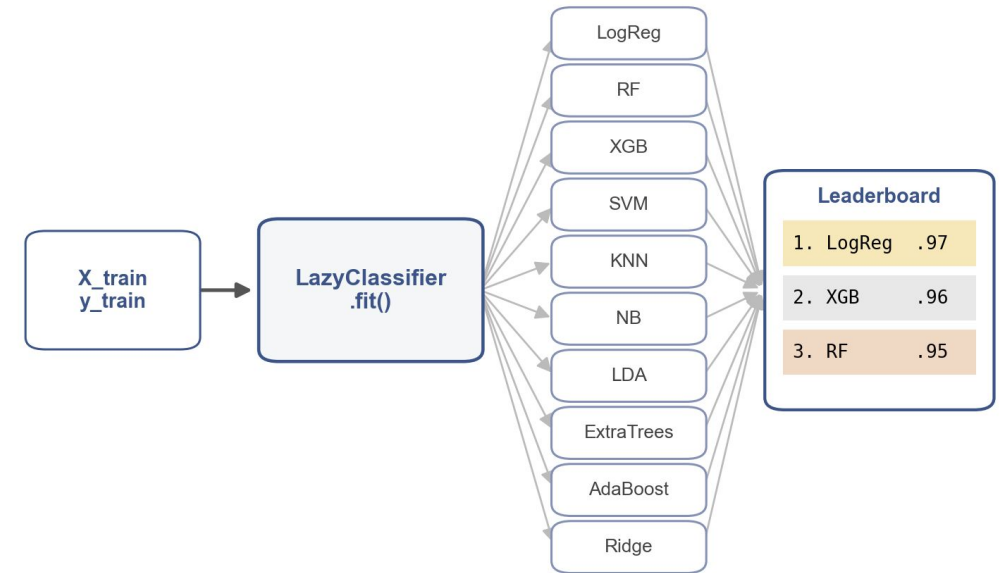
- Discrimination (AUC)  $\neq$  calibration
- Calibrated: among patients given 0.8, ~80% truly have disease
- Reliability diagram: predicted probability vs observed frequency
- RF/boosting often miscalibrated; LogReg usually well calibrated
- Fixes: Platt scaling, isotonic regression (on validation data)
- Clinical decisions use the probability - so it must mean something



### 3. LazyPredict: Automated Model Screening

---

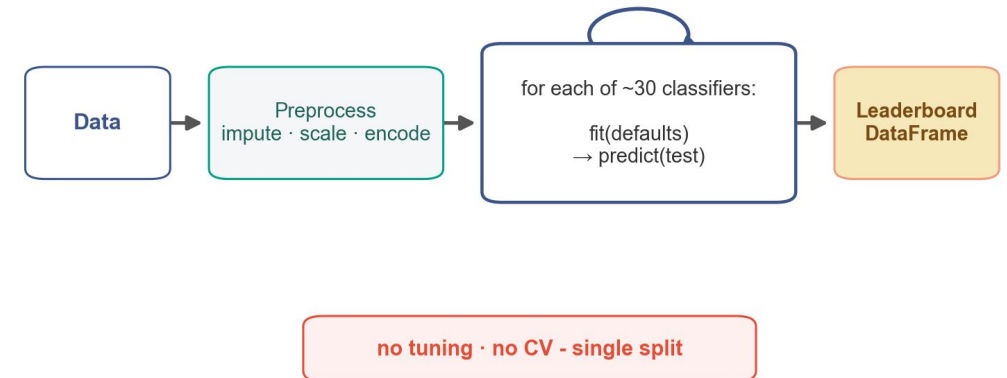
- Question every project starts with: WHICH algorithm?
- LazyPredict: fits ~30 sklearn classifiers in a few lines
- Returns a sorted leaderboard: accuracy, balanced acc., ROC-AUC, F1, time
- Purpose: fast triage of model FAMILIES, not final answers
- Finally, an algorithm with a work ethic we can all relate to
- `pip install lazypredict`



## 3.1 How LazyPredict Works

---

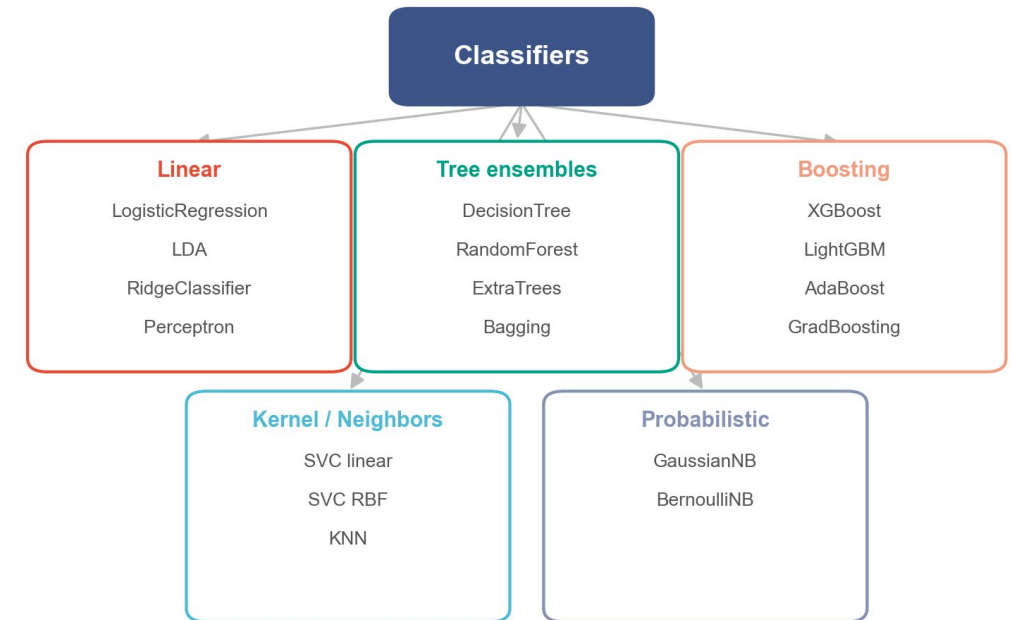
- `clf = LazyClassifier(); models, _ = clf.fit(X_tr, X_te, y_tr, y_te)`
- Under the hood: loop over sklearn classifier registry
- Each model: default hyperparameters, same train/test split
- Automatic preprocessing: imputation + scaling + encoding pipeline
- Output: one pandas DataFrame, one row per model
- Minutes of compute replace days of manual boilerplate



## 3.2 Which Models Does It Screen?

---

- Linear: Logistic Regression, LDA, Ridge, Perceptron
- Trees & ensembles: Decision Tree, Random Forest, ExtraTrees, AdaBoost, Bagging
- Boosting: XGBoost, LightGBM (if installed)
- Kernel & neighbors: SVM (linear/RBF), KNN
- Probabilistic: Gaussian & Bernoulli Naive Bayes
- Each family has a niche: linear for  $p \gg n$ , trees for interactions



## 3.3 Reading the Leaderboard

- Sort by Balanced Accuracy ( (sensitivity + specificity) / 2) or ROC-AUC - ignore raw Accuracy
- Look at model FAMILIES in the top 5, not the single winner
- Tiny score gaps ( $\pm 0.02$ ) are noise on one split - don't over-read
- Time Taken column matters at 20,000 features
- Winners  $\rightarrow$  shortlist for proper nested-CV tuning
- A leaderboard is a hypothesis generator, not a result

*ignore*

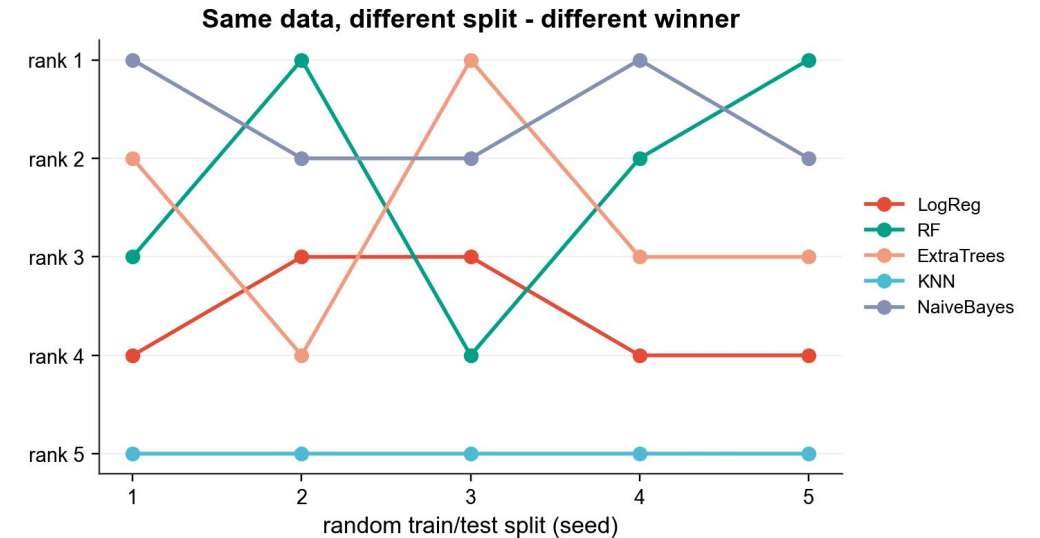
| Model              | Accuracy | Balanced Acc | ROC AUC | F1   | Time (s) |
|--------------------|----------|--------------|---------|------|----------|
| LogisticRegression | 0.93     | 0.95         | 0.98    | 0.94 | 0.04     |
| LGBMClassifier     | 0.92     | 0.94         | 0.97    | 0.93 | 0.31     |
| XGBClassifier      | 0.92     | 0.93         | 0.97    | 0.93 | 0.25     |
| RandomForest       | 0.90     | 0.92         | 0.96    | 0.91 | 0.48     |
| ExtraTrees         | 0.90     | 0.91         | 0.95    | 0.90 | 0.44     |
| SVC                | 0.88     | 0.89         | 0.94    | 0.89 | 8.72     |
| KNN                | 0.85     | 0.84         | 0.90    | 0.86 | 0.02     |
| GaussianNB         | 0.82     | 0.78         | 0.86    | 0.83 | 0.01     |

*top 3  $\rightarrow$  candidates for nested-CV tuning*

*slow*

## 3.4 LazyPredict: Limitations and Fixes

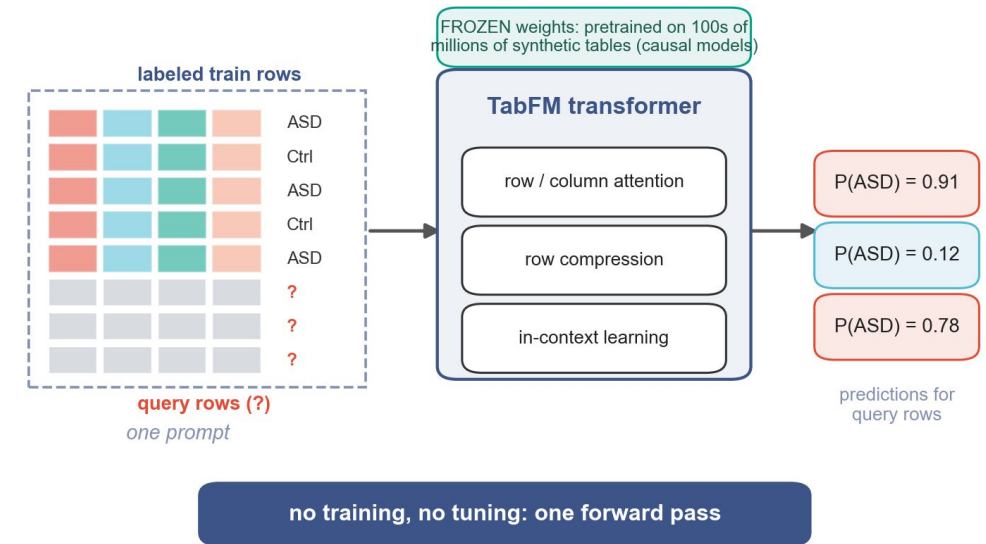
- Default hyperparameters only → underestimates tunable models
- Single train/test split → ranking unstable for small n
- No feature selection inside → screens on YOUR feature set
- Tip: Use it for triage, then nested CV on the top 3-5
- Our notebooks run the original LazyClassifier on the ASD data



## 3.5 TabFM: Google's Tabular Foundation Model

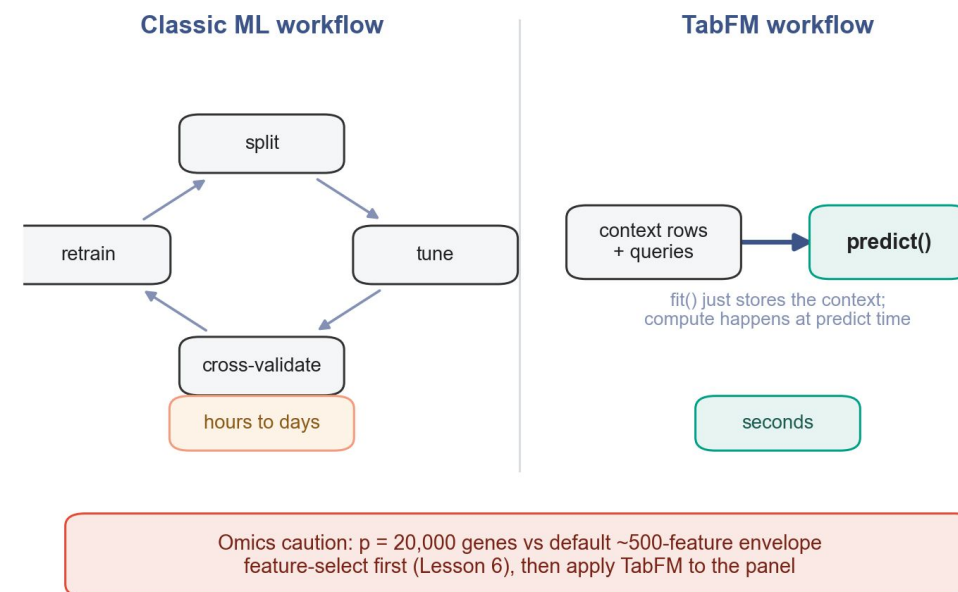
- Foundation model for tables (Google Research, June 2026)
- Zero-shot: NO training on your data - in-context learning
- Whole table (train rows + query rows) = one prompt, one forward pass
- Pretrained on 100s of millions of SYNTHETIC datasets (causal models)
- Architecture: row/column attention + row compression + ICL transformer
- In-context learning (ICL) - model learns and solves new tasks based on input-output examples provided in the prompt (context), without updating its underlying permanent model weights

TabFM: zero-shot tabular prediction by in-context learning



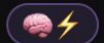
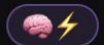
## 3.6 TabFM in Practice - and Does It Fit Omics?

- sklearn-compatible: `TabFMClassifier().fit(X, y).predict(X_new)`
- `fit()` stores context; compute happens at predict time
- Our ASD run: zero-shot test AUC 0.983 - beats every nested-CV-tuned classical model (best: XGBoost 0.92)
- Omics caution: ~500-feature envelope vs  $p = 20,000$ 
  - → feature-select first (Lesson 6): top-10 panel, AUC 0.985



## 3.7 TabArena: The Living Benchmark for Tabular ML

- [tabarena.ai](https://tabarena.ai): 51 curated datasets, continuously maintained leaderboard
- Metrics: ROC AUC (binary), log-loss (multiclass), RMSE (regression)
- Aggregation: Elo rating - models 'play matches' on every dataset
- Top 5 today: Limix-2, TabPFN-3, TabPFN-2.6,
- Caveat: Elo is pool-relative - never compare across leaderboards

| Type                                                                                | Model                                            | Overall | What it costs |              |
|-------------------------------------------------------------------------------------|--------------------------------------------------|---------|---------------|--------------|
|                                                                                     |                                                  |         | Fit (s/1K)    | Infer (s/1K) |
|  | <a href="#">LimiX-2</a> (default) ✓ \$ 🚀         | 🏆 1935  | 30.94         | 9.028        |
|  | <a href="#">TabPFN-3.5</a> (default) ✓ \$ 🚀      | 🏆 1857  | 1.84          | 0.473        |
|  | <a href="#">Causilo</a> (default) ✓ \$ 🚀         | 🏆 1783  | 0.68          | 0.104        |
|  | <a href="#">TabPFN-3.5-Fast</a> (default) ✓ \$ 🚀 | 1772    | 0.72          | 0.152        |
|  | <a href="#">TabFM</a> (default) ✓ \$ 🚀           | 1771    | 16.12         | 6.653        |

## 3.8 LimiX-2: Model the Table, Not Just One Target

- LimiX-2 (Stable AI + Tsinghua, Sept 2026): 400M-param open model
- New paradigm: Contextual Mechanism Networks (CMN)
- PFN/TabFM learn  $p(y \mid x, \text{context})$  - one designated target
- LimiX-2 learns  $p(x, y \mid \text{context})$  - the table's joint structure
- Pretraining: Context-Conditional Masked Modeling on SCM data
- One model: classification + regression + imputation + causal recovery
- 400-million-parameter structured-data foundation model

**Contextual Mechanism Networks: model the TABLE, not one target**

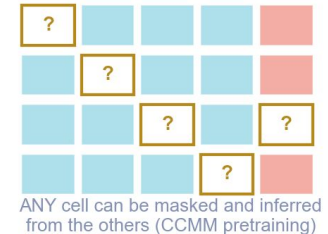
**PFN / TabFM paradigm**

$p(y \mid x, \text{context})$  - target-centric



**LimiX-2: Contextual Mechanism Network**

$p(x, y \mid \text{context})$  - joint structure of the table

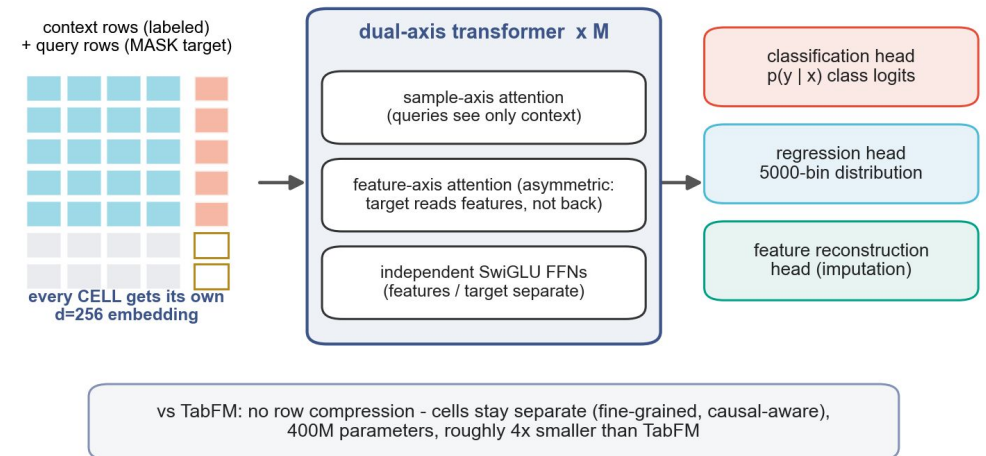


one pretrained model = classification + regression + missing-value imputation  
+ causal skeleton recovery, with zero task-specific training

## 3.9 LimiX-2: Architecture and Data Flow

- Every CELL gets its own  $d=256$  embedding - no row compression
  - (TabFM compresses rows; LimiX-2 keeps cells separate)
- Dual-axis transformer: sample-axis + feature-axis attention
  - queries attend only to context; target reads features, not back
- Low-rank column codes give features an identity; MASK = learnable vector
- Three heads: class logits, 5000-bin regression, feature reconstruction

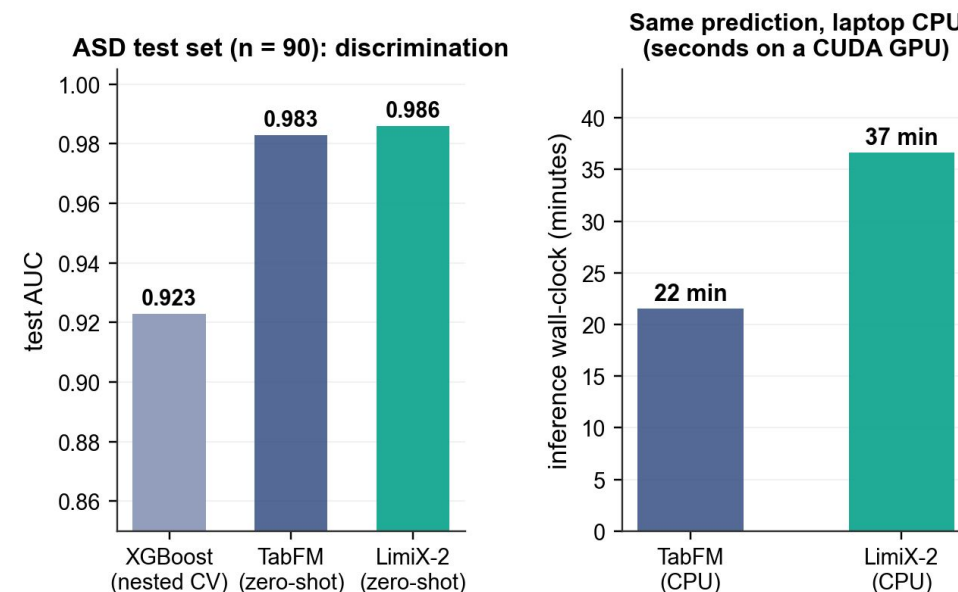
LimiX-2 data flow: cell-level embeddings through dual-axis attention



## 3.10 LimiX-2 vs TabFM on Our ASD Cohort

- Same task, zero training for both: 210 context rows, 90 queries, 152 features
- TabFM: AUC 0.983, bal.acc 0.925, ~22 min laptop CPU
- LimiX-2: AUC 0.986, bal.acc 0.933, ~37 min laptop CPU
  - (seconds on a CUDA GPU; at n=90 the AUC gap is noise)
- Both beat the nested-CV-tuned XGBoost (0.923) with zero tuning
- Takeaway: two FMs, two paradigms - benchmark on YOUR cohort

LimiX-2 vs TabFM on our ASD cohort (152 features, zero training for both)



# Lesson 5 Summary

---

- Split first, stratify always, guard the test set with your life
- Cross-validate for stability; nest it when you tune
- Overfitting is the default at  $p \gg n$  - regularize
- Standardize features; fit the scaler on training data only
- No single metric: AUC + PR + calibration tell the full story
- Screen broadly (LazyPredict, TabFM, LimiX-2), tune narrowly
- Next lesson: taming 20,000 features - feature selection



## Exercise - how stable is model screening?

---

- Re-run the LazyPredict screening cell three times with different random\_state values in train\_test\_split (e.g. 1, 7, 99)
- Record the top-5 models each time
- Then answer in 3-4 sentences: which models stay on the podium, which shuffle around, and what does this imply about choosing a model from a single 90-sample test split?